

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-aea6e490be731bbfd.jsonl`  
- SHA-256: `bb9171ca72ca6e0eb5c219d47bdfdf10927f52440f50b1ca66c38a55270912e06`  
- Source modified: `2026-06-10T06:15:05+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:12:46.619Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("eng-pipeline") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"Detector artifacts keyed by file basename, not video_id ? same-stem videos can silently reuse the wrong cached frames and produce a trajectory for the wrong video","severity":"high","area":"detectors (WASB/TrackNet)","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_infer.py:361-366","evidence":"wasb_runner.run_predict stages to ~/clips/<basename> and uses frames_dir=~<clips/<stem>_frames and output output/wasb/<stem>_wasb.csv (wasb_runner.py:87,122,114); tracknet_runner uses <stem>_predict.csv in output/tracknet. wasb_infer.extract_frames skips re-extraction when 'existing >= expected' PNGs are present (wasb_infer.py:361-366) ? the frames dir is shared per-stem, so processing B/match.mp4 after A/match.mp4 (different files, same basename, B's frame count <= A's) runs WASB on A's leftover frames. The cache manifest does compare frames_total (manifest_compatible, :130-141) so the detector cache resets, but the FRAMES themselves are the wrong video's. The staged video at ~/clips/match.mp4 is overwritten by cp, but extract_frames never decodes it because the count check passes.", "why_it_matters":"Phone/camera filenames collide constantly (VID_0001.mp4, match.mp4); the result is a trajectory, candidate windows, and persisted candidate_segments computed from a different video with no error anywhere ? exactly the kind of silent training-data poisoning the candidate table is meant to feed fine-tuning. Everything else in the system is keyed by MD5; the detector layer is the one place keyed by name. NEW finding."} Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and

a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:12:54.971Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36     python wasb_infer.py --frames_dir /path/frames --weights
37     ../pretrained_weights/wasb_badminton_best.pth.tar \
38     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39     every 1000] [--fresh]
40     """
41     import os
42     import os.path as osp
43     import csv
44     import glob
45     import json
46     import argparse
47     import logging
48     from collections import defaultdict
49
50     logger = logging.getLogger(__name__)
51
52     CACHE_VERSION = 1
53     _DET_FILE = "det_raw.jsonl"
54     _MANIFEST_FILE = "manifest.json"
55
56     def _build_cfg(weights: str, sport: str):
57         """Compose the WASB eval config via Hydra, overriding model/weights/device."""
58         from hydra import compose, initialize
59         with initialize(config_path="configs", version_base=None):
60             cfg = compose(config_name="eval", overrides=[
```

```

60         f"dataset={sport}",
61         "model=wasb",
62         f"detector.model_path={weights}",
63         "runner.device=cuda",
64         "runner.gpus=[0]",          # this box has a single GPU; default config assumes
more
65     ])
66     return cfg
67
68
69     def _frame_id(path: str) -> int:
70         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
71         stem = osp.splitext(osp.basename(path))[0]
72         digits = "".join(ch for ch in stem if ch.isdigit())
73         return int(digits) if digits else -1
74
75
76     # ----- #
77     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
78     # ----- #
79     def _cache_paths(cache_dir: str):
80         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
81
82
83     def default_cache_dir(out_csv: str) -> str:
84         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
85         d = osp.dirname(osp.abspath(out_csv))
86         stem = osp.splitext(osp.basename(out_csv))[0]
87         return osp.join(d, stem + "_wasbcache")
88
89
90     def _atomic_write_text(path: str, text: str) -> None:
91         """Write then rename so a crash mid-write can't leave a half-written file."""
92         tmp = path + ".tmp"
93         with open(tmp, "w") as f:
94             f.write(text)
95             f.flush()
96             os.fsync(f.fileno())
97         os.replace(tmp, path)
98
99
100     def _det_plain(det) -> list:
101         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
102         xy = det["xy"]
103         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
104
105
106     def _dets_to_tracker(plain_list):
107         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
108         the tracker does vector math / np.linalg.norm on it)."""
109         import numpy as np
110         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
111                 for p in plain_list]
112
113
114     def write_manifest(cache_dir: str, manifest: dict) -> None:
115         _, mpath = _cache_paths(cache_dir)
116         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
117
118
119     def read_manifest(cache_dir: str):
120         _, mpath = _cache_paths(cache_dir)

```

```

121     if not osp.exists(mpath):
122         return None
123     try:
124         with open(mpath) as f:
125             return json.load(f)
126     except (json.JSONDecodeError, OSError):
127         return None
128
129
130 def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
131                        frames_in: int, frames_total: int) -> bool:
132     """A cache is reusable only if the run that produced it matches this run's
133     inputs."""
134     if not manifest or manifest.get("version") != CACHE_VERSION:
135         return False
136     return (
137         manifest.get("frames_dir") == osp.abspath(frames_dir)
138         and manifest.get("weights") == weights
139         and manifest.get("sport") == sport
140         and manifest.get("frames_in") == frames_in
141         and manifest.get("frames_total") == frames_total
142     )
143
144 def reset_cache(cache_dir: str) -> None:
145     """Drop any existing cache so the next run starts clean."""
146     det_jsonl, mpath = _cache_paths(cache_dir)
147     for p in (det_jsonl, mpath):
148         if osp.exists(p):
149             os.remove(p)
150
151
152 def load_and_clean_cache(cache_dir: str):
153     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
154     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
155     prefix so a trailing half-written line from a crash can't corrupt future appends.
156
157     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
158     [x, y, score, scale] candidates (accumulated across every window the frame is in,
159     in window order ? identical to a non-resumed run).
160     """
161     det_jsonl, _ = _cache_paths(cache_dir)
162     det_by_fid = defaultdict(list)
163     valid = []
164     if osp.exists(det_jsonl):
165         with open(det_jsonl, "r") as f:
166             for line in f:
167                 s = line.strip()
168                 if not s:
169                     continue
170                 try:
171                     obj = json.loads(s)
172                 except json.JSONDecodeError:
173                     break # truncated trailing line (crash mid-
flush)
174                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
175                     break
176                 valid.append(s)
177                 for fid, plain in obj["f"]:
178                     det_by_fid[int(fid)].extend([list(p) for p in plain])
179     # Guarantee a clean append boundary by rewriting only the good prefix.
180     _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
181     return len(valid), det_by_fid
182
183

```

```

184     def _append_windows(fh, objs) -> None:
185         """Append window records as JSONL and durably flush (bounded loss on crash)."""
186         for o in objs:
187             fh.write(json.dumps(o, separators=(",", ":")) + "\n")
188         fh.flush()
189         os.fsync(fh.fileno())
190
191
192     def _atomic_write_trajectory(out_csv: str, rows) -> None:
193         os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
194         tmp = out_csv + ".tmp"
195         with open(tmp, "w", newline="") as f:
196             w = csv.writer(f)
197             w.writerow(["Frame", "Visibility", "X", "Y"])
198             w.writerows(rows)
199             f.flush()
200             os.fsync(f.fileno())
201         os.replace(tmp, out_csv)
202
203
204     # ----- #
205     # Inference
206     # ----- #
207     def run(frames_dir: str, weights: str, out_csv: str, sport: str = "badminton",
208             limit: int = 0, batch_size: int = 8, num_workers: int = 4,
209             log_every_batches: int = 50, cache_dir: str = None,
210             flush_every: int = 1000, fresh: bool = False) -> str:
211         import time
212         import torch
213         from torch.utils.data import DataLoader
214         from detectors import build_detector
215         from trackers import build_tracker
216         from dataloaders import build_img_transforms, build_seq_transforms
217         from dataloaders.dataset_loader import ImageDataset
218         from utils import Center
219
220         cfg = _build_cfg(weights, sport)
221         frames_in = int(cfg["model"]["frames_in"])
222         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
223         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
224
225         frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
226         glob.glob(osp.join(frames_dir, "*.jpg")))
227         if limit:
228             frames = frames[:limit]
229         if len(frames) < frames_in:
230             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in
231 {frames_dir}")
232
233         # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
234         checkpoint).
235         samples = []
236         for i in range(len(frames) - frames_in + 1):
237             window = frames[i:i + frames_in]
238             annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
239 for p in window]
240             samples.append({"frames": window, "annos": annos})
241         windows_total = len(samples)
242
243         # ---- cache setup / resume decision ----- #
244         if cache_dir is None:
245             cache_dir = default_cache_dir(out_csv)
246         os.makedirs(cache_dir, exist_ok=True)
247         det_jsonl, _ = _cache_paths(cache_dir)
248         manifest = None if fresh else read_manifest(cache_dir)

```

```

245     resume = manifest_compatible(
246         manifest or {}, frames_dir=frames_dir, weights=weights, sport=sport,
247         frames_in=frames_in, frames_total=len(frames),
248     )
249     if resume:
250         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
251         logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
252     else:
253         if manifest is not None:
254             logger.info("cache incompatible with this run -> starting fresh")
255             reset_cache(cache_dir)
256             windows_done, det_by_fid = 0, defaultdict(list)
257
258     base_manifest = {
259         "version": CACHE_VERSION,
260         "frames_dir": osp.abspath(frames_dir),
261         "weights": weights,
262         "sport": sport,
263         "frames_in": frames_in,
264         "frames_total": len(frames),
265         "windows_total": windows_total,
266         "windows_done": windows_done,
267     }
268     write_manifest(cache_dir, base_manifest)
269
270     # ---- detector pass over the un-cached windows ----- #
271     remaining = samples[windows_done:]
272     if remaining:
273         _, transform_test = build_img_transforms(cfg)
274         try:
275             _, seq_transform_test = build_seq_transforms(cfg)
276         except Exception:
277             seq_transform_test = None
278         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
279                         transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
280         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=num_workers)
281         detector = build_detector(cfg)
282
283         t0 = time.time()
284         done = windows_done
285         win_idx = windows_done
286         log_every = max(1, log_every_batches)
287         flush_n = max(1, flush_every)
288         pending = []
289         logger.info(f"detecting on {len(remaining)} remaining windows "
290                 f"(total {windows_total}, batch={batch_size},
workers={num_workers})...")
291         det_f = open(det_jsonl, "a")
292         try:
293             with torch.no_grad():
294                 for bi, batch in enumerate(loader):
295                     imgs, _hms, trans = batch[0], batch[1], batch[2]
296                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
297                     batch_results, _ = detector.run_tensor(imgs, trans)
298                     nb = imgs.shape[0]
299                     for ib in range(nb):
300                         frames_payload = []
301                         for ie in sorted(batch_results[ib].keys()):
302                             p = img_paths[ie][ib]
303                             fid = _frame_id(p)
304                             plain = [_det_plain(d) for d in batch_results[ib][ie]]

```

```

305         frames_payload.append([fid, plain])
306         det_by_fid[fid].extend(plain)
307         pending.append({"w": win_idx, "f": frames_payload})
308         win_idx += 1
309     done += nb
310     if len(pending) >= flush_n or done >= windows_total:
311         _append_windows(det_f, pending)
312         pending = []
313         base_manifest["windows_done"] = done
314         write_manifest(cache_dir, base_manifest)
315     if (bi + 1) % log_every == 0 or done >= windows_total:
316         el = time.time() - t0
317         new_done = done - windows_done
318         rate = new_done / el if el > 0 else 0.0
319         eta = (windows_total - done) / rate if rate > 0 else 0.0
320         logger.info(f"{done}/{windows_total} "
321                    f"({100 * done // max(1, windows_total)}%) |
322 {rate:.1f} win/s | "
323                    f"elapsed {el:.0f}s | ETA {eta:.0f}s")
324     if pending:
325         _append_windows(det_f, pending)
326         base_manifest["windows_done"] = done
327         write_manifest(cache_dir, base_manifest)
328     finally:
329         det_f.close()
330     logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
331 else:
332     logger.info("detector cache already complete -> tracker-only re-run")
333
334 # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
335 tracker = build_tracker(cfg)
336 tracker.refresh()
337 rows = []
338 for fid in sorted(det_by_fid.keys()):
339     r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
340     rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
341
342 _atomic_write_trajectory(out_csv, rows)
343 visible = sum(1 for _, v, *_ in rows if v)
344 logger.info(f"wrote {len(rows)} rows -> {out_csv}")
345 logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
346 return out_csv
347
348 def extract_frames(video_path: str, frames_dir: str) -> str:
349     """Decode a video to PNG frames named by source frame index (00000000.png ...).
350
351     Idempotent: if the dir already holds the expected number of frames (matching the
352     container's reported frame count) we skip re-decoding ? re-extraction after a
353     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
354     extracting with ffmpeg on the host and passing --frames_dir.
355     """
356     import cv2
357     os.makedirs(frames_dir, exist_ok=True)
358     cap = cv2.VideoCapture(video_path)
359     if not cap.isOpened():
360         raise SystemExit(f"cannot open video: {video_path}")
361     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
362     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
363     if expected > 0 and existing >= expected:
364         cap.release()
365         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
366         return frames_dir
367     i = 0
368     while True:

```

```

369         ok, frame = cap.read()
370         if not ok:
371             break
372         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
373         i += 1
374     cap.release()
375     logger.info(f"extracted {i} frames -> {frames_dir}")
376     return frames_dir
377
378
379 def main():
380     ap = argparse.ArgumentParser()
381     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
382     ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
383     ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
384     ap.add_argument("--weights", required=True)
385     ap.add_argument("--out", required=True)
386     ap.add_argument("--sport", default="badminton")
387     ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
388     ap.add_argument("--batch-size", type=int, default=8)
389     ap.add_argument("--num-workers", type=int, default=4)
390     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
391     ap.add_argument("--cache-dir", default=None,
392                     help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
393     ap.add_argument("--flush-every", type=int, default=1000,
394                     help="fsync the detector cache every N windows (bounds reboot
loss)")
395     ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
396     args = ap.parse_args()
397
398     frames_dir = args.frames_dir
399     if args.video:
400         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
401     if not frames_dir:
402         raise SystemExit("provide --frames_dir or --video")
403     run(frames_dir, args.weights, args.out, args.sport, args.limit,
404         batch_size=args.batch_size, num_workers=args.num_workers,
405         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
406         flush_every=args.flush_every, fresh=args.fresh)
407
408
409 if __name__ == "__main__":
410     main()
411

```

3. user (2026-06-10T06:12:55.342Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5     2. copies wasb_infer.py into the WASB repo's src/ dir,
6     3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7     4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """

```

```

12     import os
13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22     from backend.pipeline.detectors.base import DetectorRunner
23
24     logger = logging.getLogger(__name__)
25
26     # Windows path to the inference wrapper we stage into WSL.
27     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 0 # 0 = no timeout
41
42         @classmethod
43         def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
44             w = (idx_cfg or {}).get("wasb", {}) or {}
45             return cls(
46                 distro=w.get("wsl_distro", "Ubuntu"),
47                 repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
48                 conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
49                 conda_env=w.get("conda_env", "wasb"),
50                 weights_path=w.get("weights_path",
51                                     "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
52                 sport=w.get("sport", "badminton"),
53                 wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
54                 timeout_sec=int(w.get("timeout_sec", 0)),
55             )
56
57
58     class WasbRunner(DetectorRunner): # DetectorRunner ABC (finish item 6 / new item 1)
59         def __init__(self, cfg: WasbConfig):
60             self.cfg = cfg
61
62         def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
63             full = f"source {self.cfg.conda_sh} && {inner_cmd}"
64             argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
65             logger.debug("WSL exec: %s", full)
66             return subprocess.run(argv, capture_output=True, text=True,
67                                   timeout=(self.cfg.timeout_sec or None))
68
69         def healthcheck(self) -> Tuple[bool, str]:
70             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
71             cmd = (f"conda activate {self.cfg.conda_env} && "
72                   f"python -c \"import torch; print('torch', torch.__version__, "

```

```

73         f"'cuda', torch.cuda.is_available())\")
74     try:
75         res = self._wsl_bash(cmd)
76     except FileNotFoundError:
77         return False, "`wsl` not found ? Windows + WSL2 required."
78     except subprocess.TimeoutExpired:
79         return False, "WSL healthcheck timed out."
80     out = (res.stdout or "").strip()
81     if res.returncode != 0:
82         return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
83     return True, out
84
85     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
86         src_mnt = to_wsl_mnt_path(video_win_path)
87         dest = f"{self.cfg.wsl_stage_dir}/{base}"
88         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
89 {shlex.quote(dest)} && echo OK"
90     try:
91         res = self._wsl_bash(cmd)
92     except subprocess.TimeoutExpired:
93         logger.error("Staging video into WSL timed out.")
94         return None
95     if res.returncode != 0 or "OK" not in (res.stdout or ""):
96         logger.error("Failed to stage video into WSL: %s", (res.stderr or
97 res.stdout).strip())
98         return None
99     return dest
100
101     def _stage_infer_script(self) -> bool:
102         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
103         src_mnt = to_wsl_mnt_path(_INFER_WIN)
104         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
105 OK"
106         res = self._wsl_bash(cmd)
107         return res.returncode == 0 and "OK" in (res.stdout or "")
108
109     def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
110         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or
111 None."""
112         os.makedirs(output_win_dir, exist_ok=True)
113         # Normalize for cross-platform basename (tests use Windows paths on Linux).
114         video_win_path = str(video_win_path).replace("\\", "/")
115         base = os.path.basename(video_win_path)
116         stem = os.path.splitext(base)[0]
117         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
118         expected_csv_win = os.path.join(output_win_dir, f"{stem}_wasb.csv")
119
120         if not self._stage_infer_script():
121             logger.error("Could not stage wasb_infer.py into WSL.")
122             return None
123         staged_video = self._stage_video(video_win_path, base)
124         if staged_video is None:
125             return None
126         frames_dir = f"{self.cfg.wsl_stage_dir}/{stem}_frames"
127
128         cmd = (
129             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
130             f"python wasb_infer.py "
131             f"--video {shlex.quote(staged_video)} "
132             f"--frames_out_dir {shlex.quote(frames_dir)} "
133             f"--weights {shlex.quote(self.cfg.weights_path)} "
134             f"--sport {shlex.quote(self.cfg.sport)} "
135             f"--out {shlex.quote(wsl_out_dir + '/' + stem + '_wasb.csv')}"
136         )
137         logger.info("Running WASB inference on %s ...", base)

```

```

134         try:
135             res = self._wsl_bash(cmd)
136         except subprocess.TimeoutExpired:
137             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
138             return None
139         if res.returncode != 0:
140             logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
141             return None
142         if not os.path.exists(expected_csv_win):
143             logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
144             return None
145             logger.info("WASB trajectory CSV: %s", expected_csv_win)
146             return expected_csv_win
147
148         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
149         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
150         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
151

```

4. user (2026-06-10T06:13:32.011Z)

```

backend\eval\export_wasb_segments.py:140:         out_csv: str =
"output/wasb_tuned_segments.csv", labels_csv: Optional[str] = None,
backend\eval\export_wasb_segments.py:208:         ap.add_argument("--out",
default="output/wasb_tuned_segments.csv")
backend\eval\export_wasb_segments.py:216:         ap.add_argument("--clips-dir",
default="output/wasb_tuned_clips")
backend\pipeline\detectors\base.py:15:- run_predict(video_win_path: str, output_win_dir: str) ->
Optional[str]
backend\pipeline\detectors\base.py:22: Return (ok, message). Never raises. "ok" decides whether
to proceed to run_predict.
backend\pipeline\detectors\base.py:31:2. Implement run_predict and healthcheck (return (bool,
str) tuple).
backend\pipeline\detectors\base.py:77:         def run_predict(self, video_win_path: str,
output_win_dir: str) -> Optional[str]:
backend\pipeline\detectors\tracknet_runner.py:128:         def run_predict(self, video_win_path: str,
output_win_dir: str) -> Optional[str]:
backend\pipeline\detectors\tracknet_runner.py:131:             ``output_win_dir`` is a Windows
directory (under the repo's output/) that
backend\pipeline\detectors\tracknet_runner.py:142:             os.makedirs(output_win_dir,
exist_ok=True)
backend\pipeline\detectors\tracknet_runner.py:154:             wsl_out =
to_wsl_mnt_path(output_win_dir)
backend\pipeline\detectors\tracknet_runner.py:155:             expected_csv_win =
os.path.join(output_win_dir, f"{stem}_predict.csv")
backend\pipeline\detectors\wasb_runner.py:106:         def run_predict(self, video_win_path: str,
output_win_dir: str) -> Optional[str]:
backend\pipeline\detectors\wasb_runner.py:108:             os.makedirs(output_win_dir, exist_ok=True)
backend\pipeline\detectors\wasb_runner.py:113:             wsl_out_dir =
to_wsl_mnt_path(output_win_dir)
backend\pipeline\detectors\wasb_runner.py:114:             expected_csv_win =
os.path.join(output_win_dir, f"{stem}_wasb.csv")
backend\pipeline\segmenters\tracknet_hybrid.py:128:             csv_path =
runner.run_predict(video_path, tn_out_dir)
backend\pipeline\segmenters\wasb_hybrid.py:115:             csv_path = runner.run_predict(video_path,
out_dir)

```

5. user (2026-06-10T06:13:32.389Z)

```

1         """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2

```

```

3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
33      indexing.tracknet)."""
34          distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""                          # WSL path to the .h5/.keras weights
39          (REQUIRED)
40          queue_length: int = 5
41          stage_in_wsl: bool = True                        # copy video into WSL fs first
42          (perf)
43          wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
44      in WSL
45          timeout_sec: int = 0                            # 0 = no timeout
46
47
48      @classmethod
49      def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
50          tn = (idx_cfg or {}).get("tracknet", {}) or {}
51          return cls(
52              distro=tn.get("wsl_distro", "Ubuntu"),
53              repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
54              conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
55              conda_env=tn.get("conda_env", "TrackNetV4"),
56              weights_path=tn.get("weights_path", ""),
57              queue_length=int(tn.get("queue_length", 5)),
58              stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
59              wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
60              timeout_sec=int(tn.get("timeout_sec", 0)),
61          )
62
63      def to_wsl_mnt_path(win_path: str) -> str:
64          """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66          Already-POSIX paths (starting with / or ~) are returned unchanged so the
67          same helper is safe to call on either kind of path.

```

```

64     """
65     p = str(win_path)
66     if p.startswith("/") or p.startswith("~"):
67         return p
68     p = p.replace("\\", "/")
69     if len(p) >= 2 and p[1] == ":":
70         drive = p[0].lower()
71         return f"/mnt/{drive}{p[2:]}"
72     return p
73
74
75 @dataclass
76 class TrajectoryPoint:
77     frame: int
78     visible: bool
79     x: float
80     y: float
81
82
83 class TrackNetRunner(DetectorRunner): # DetectorRunner ABC (finish item 6 / new item 1)
84     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
85
86     Implements the common DetectorRunner contract so a future Linux-native or
87     alternative trajectory runner can be substituted without touching the hybrids.
88     """
89
90     def __init__(self, cfg: TrackNetConfig):
91         self.cfg = cfg
92
93     # ----- WSL exec
94
95     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
96         """Run a bash command string inside the configured WSL distro."""
97         full = f"source {self.cfg.conda_sh} && {inner_cmd}"
98         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
99         logger.debug("WSL exec: %s", full)
100         return subprocess.run(
101             argv, capture_output=True, text=True,
102             timeout=(self.cfg.timeout_sec or None),
103         )
104
105     def healthcheck(self) -> Tuple[bool, str]:
106         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
107
108         Returns (ok, message). Cheap to call before a long run.
109         """
110         cmd = (
111             f"conda activate {self.cfg.conda_env} && "
112             f"python -c \"import tensorflow as tf; "
113             f"print('TF', tf.__version__, 'GPUs', "
114             f"len(tf.config.list_physical_devices('GPU')))\n"
115         )
116         try:
117             res = self._wsl_bash(cmd)
118             except FileNotFoundError:
119                 return False, "`wsl` not found ? is this running on Windows with WSL2
120                 installed?"
121             except subprocess.TimeoutExpired:
122                 return False, "WSL healthcheck timed out."
123             out = (res.stdout or "").strip()
124             if res.returncode != 0:
125                 return False, f"WSL env check failed: {(res.stderr or out).strip()}"
126             return True, out
127
128     # ----- inference

```

```

127
128     def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
129         """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.
130
131         ``output_win_dir`` is a Windows directory (under the repo's output/) that
132         WSL writes into via its /mnt view, so the Windows process can read the CSV
133         back with no copy.
134         """
135         if not self.cfg.weights_path:
136             logger.error(
137                 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
138                 "in config.json to the WSL path of your trained TrackNetV4 weights."
139             )
140             return None
141
142         os.makedirs(output_win_dir, exist_ok=True)
143         # Normalize separators so basename/splitext work when tests (or collector)
144         # pass Windows-style paths on a Linux host.
145         video_win_path = str(video_win_path).replace("\\", "/")
146         base = os.path.basename(video_win_path)
147         stem, ext = os.path.splitext(base)
148
149         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
150         if ext.lower() not in (".mp4", ".avi"):
151             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
152             return None
153
154         wsl_out = to_wsl_mnt_path(output_win_dir)
155         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
156
157         # Resolve the video path WSL will read.
158         if self.cfg.stage_in_wsl:
159             staged = self._stage_video(video_win_path, base)
160             if staged is None:

```

6. assistant (2026-06-10T06:13:40.225Z)

Now let me check the hybrid segmenter callers to confirm the output dir is stem-keyed and not video_id-keyed, and check the docs for any documented gotcha.

7. user (2026-06-10T06:13:41.645Z)

```

1     """WASB-based hybrid play segmenter.
2
3     Identical three-phase shape to TrackNetHybridSegmenter, but Phase 2 uses **WASB**
4     (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via WSL2
5     (backend/detectors/wasb_runner.py) instead of TrackNetV4. WASB ships pretrained
6     badminton weights and runs today, so this is the live shuttle-motion substrate.
7     Phase 3 (AI semantic boundaries) and Phase 4 (DB) are shared with the other hybrids.
8
9     For the per-video *calibration* goal we mainly use Phase 2 (trajectory -> candidate
10    windows); the windowing thresholds (velocity_thresh / merge_gap / min_window_duration)
11    are what get tuned against golden rally labels.
12    """
13    import os
14    import time
15    import logging
16    import concurrent.futures
17    from typing import List, Dict, Any
18
19    import cv2
20
21    from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
22    from backend.utils.video import get_video_duration, extract_video_chunk

```

```

23     from backend.sports import sport_registry
24     from backend.providers import provider_registry
25     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
26     from backend.pipeline.detectors.base import DetectorRunner # ABC for the de-risked
detector seam
27
28     logger = logging.getLogger(__name__)
29
30
31     def _fmt_ts(seconds: float) -> str:
32         seconds = max(0, int(seconds))
33         h, rem = divmod(seconds, 3600)
34         m, s = divmod(rem, 60)
35         return f"{h:02d}:{m:02d}:{s:02d}"
36
37
38     class WasbHybridSegmenter(BasePlaySegmenter):
39         @property
40         def name(self) -> str:
41             return "wasb_hybrid"
42
43         @property
44         def description(self) -> str:
45             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
46                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
47
48         @staticmethod
49         def _probe_fps_width(video_path: str) -> tuple:
50             cap = cv2.VideoCapture(video_path)
51             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
52             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
53             cap.release()
54             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
55
56         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
57                         max_frames: int = None) -> List[Dict[str, Any]]:
58             sport_name = self.config.get("sport", "badminton")
59             try:
60                 sport_profile = sport_registry.get(sport_name)
61             except KeyError as e:
62                 logger.error(str(e))
63                 return []
64
65             idx_cfg = self.config.get("indexing", {})
66             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
67             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
68
69             api_key_env_var = f"{provider_name.upper()}_API_KEY"
70             api_key = os.environ.get(api_key_env_var)
71             if not api_key:
72                 logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")
73                 return []
74             try:
75                 provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
76             except KeyError as e:
77                 logger.error(str(e))
78                 return []
79

```

```

80     video_duration = get_video_duration(video_path)
81     if video_duration <= 0:
82         logger.error("Invalid video duration.")
83         return []
84     fps, frame_width = self._probe_fps_width(video_path)
85
86     wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
87     runner: DetectorRunner = WasbRunner(wasb_cfg)
88
89     w = idx_cfg.get("wasb", {})
90     velocity_thresh = w.get("velocity_threshold", 0.02)
91     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
92     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
93     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
94     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
95     log_candidates = idx_cfg.get("log_yolo_candidates", True)
96     store_candidates = idx_cfg.get("store_yolo_candidates", True)
97
98     detection_params = {
99         "detector": "wasb_hybrid",
100         "velocity_thresh": velocity_thresh,
101         "merge_gap": merge_gap,
102         "min_action_window_duration": min_window_duration,
103         "weights_path": wasb_cfg.weights_path,
104     }
105
106     ok, msg = runner.healthcheck()
107     if not ok:
108         logger.error("WASB WSL environment not ready: %s", msg)
109         return []
110     logger.info("WASB WSL env OK: %s", msg)
111
112     # --- Phase 2: shuttle trajectory -> candidate rally windows ---
113     t_start = time.time()
114     out_dir = os.path.join("output", "wasb")
115     csv_path = runner.run_predict(video_path, out_dir)
116     if not csv_path:
117         logger.error("WASB produced no trajectory; aborting.")
118         return []
119
120     points = runner.parse_trajectory_csv(csv_path)
121     logger.info("Parsed %d trajectory points (%d visible).",
122               len(points), sum(1 for p in points if p.visible))
123
124     all_candidate_windows = runner.trajectory_to_action_windows(
125         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
126         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
127         min_window_duration=min_window_duration, chunk_index=0,
128     )
129     logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
130               time.time() - t_start, len(all_candidate_windows))
131
132     if log_candidates:
133         for cw in all_candidate_windows:
134             logger.info(f"[WASB rally] {_fmt_ts(cw['start'])} - {_fmt_ts(cw['end'])} "
135                       f"({cw['end'] - cw['start']:.1f}s) | "
136                       f"peak_v={cw['peak_velocity']:.3f} "
137                       f"mean_v={cw['mean_velocity']:.3f}")
138
139     candidate_ids = [None] * len(all_candidate_windows)
140     if store_candidates:
141         for i, cw in enumerate(all_candidate_windows):
142             stats = {
143                 "peak_velocity": cw["peak_velocity"], "mean_velocity":

```

```

cw["mean_velocity"],
143             "active_frame_count": cw["active_frame_count"], "track_id_count":
cw["track_id_count"],
144         }
145         candidate_ids[i] = self.db.add_candidate_segment(
146             video_id, detector="wasb_hybrid", start_time=cw["start"],
end_time=cw["end"],
147             chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
148             stats=stats, detection_params=detection_params,
149         )
150
151         if not all_candidate_windows:
152             return []
153
154         # --- Phase 3: AI provider handoff (shared pattern) ---
155         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
156         os.makedirs(handoff_dir, exist_ok=True)
157         logger.info("Phase 3: %s validation on %d candidate windows...",
158             provider_name, len(all_candidate_windows))
159
160         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:

```

8. user (2026-06-10T06:13:43.330Z)

5:**Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this doc).
Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).

78:## 2. SUBSYSTEMS

121:[Omitted long matching line]

122:[Omitted long matching line]

123:[Omitted long matching line]

131:- `@backend/eval/calibrate\_local.py` ? distill Gemini->local thresholds (windowing + net-crossing gate, no cloud at runtime). `::local\_grid` (180 cfs), `::make\_local\_predict\_fn` (adds `rally\_gate.apply\_gate` when `min\_crossings>0`), `::build\_videos` (? `SystemExit` if a labels file yields no rallies), `::run`/`::main`. \$
`LocalConfig=(velocity,merge,min\_dur,min\_crossings)`, `BASELINE\_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.

141:[Omitted long matching line]

292:## 8. TEST MAP (`tests/test\_\*.py` -> subsystem)

9. user (2026-06-10T06:13:55.010Z)

115

116 ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)

117 - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() -> Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on threading health into reports. Re-exported from `detectors/\_\_init\_\_.py`. This finishes the WSL de-risk seam (low-regret 6 / review item 1).

118 - `@backend/pipeline/detectors/tracknet\_runner.py`

119 - `::TrackNetConfig` (+`.from\_indexing\_cfg`; ? key drift `wsl\_distro` JSON -> `distro` field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.

120 - `::to\_wsl\_mnt\_path` (`C:\x`->`/mnt/c/x`; ? POSIX/~` passthrough, UNC unhandled), `::TrajectoryPoint` (frame,visible,x,y).

121 - `::parse\_trajectory\_csv` @staticmethod** + `::trajectory\_to\_action\_windows` @staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights\_path` defaults `` -> `run\_predict` returns None; predict.py only `mp4/.avi` + hard-names
`<stem>\_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`,
`frame\_width<=0->1280` silent; `track\_id\_count` hardcoded 1; `timeout\_sec=0` -> NO timeout (hangs forever). `::stage\_video` "OK" sentinel.

122 - `@backend/pipeline/detectors/wasb\_runner.py::WasbRunner` (inherits DetectorRunner) (+`::WasbConfig`, from `indexing.wasb`; ? `weights\_path` HAS a real default
`wasb\_badminton\_best.pth.tar` ? no required guard) ? stages `wasb\_infer.py` (`::INFER\_WIN`) +

```

video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` =
`staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-
named `_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until
staged); `wsl_bash` duplicated (drift risk). Both runners now satisfy the DetectorRunner ABC
for future swappability.
123     - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT
repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache:
`::DET_FILE='det_raw.jsonl'`, `::MANIFEST_FILE='manifest.jsonl'`, `::CACHE_VERSION=1`,
`::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to
heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights`
compared raw), `::atomic_write_text/_atomic_write_trajectory`, `::dets_to_tracker` (xy MUST be
np.array), `::build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW),
`::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker
STATEFUL -> always fully re-run; `--fresh` ignores cache.
124     - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure,
no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'),
`::count_net_crossings` (sign changes vs median net; deadband jitter zone),
`::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds).
`::GatedWindow` (start,end,crossings,visible_points,kept). ? window contract here is
`(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local /
distill_local / export_wasb_segments.
125
126     ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
127     - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`,
`::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape),
`::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN)
for mean_iou/boundary errors.
128     - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end`
CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
129     - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video
k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video;
needs ?? labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT),
`::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. $
`PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap
>~0.1` = overfit alarm.
130     - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
131     - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing +
net-crossing gate, no cloud at runtime). `::local_grid` (180 cfgs), `::make_local_predict_fn`
(adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a
labels file yields no rallies), `::run`/`::main`. $
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
132     - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison
+ optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video),
`::SEG_FIELDS`, `::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::maybe_slice`
(lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads
straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is
load-bearing.
133     - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in
run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1),
`::default_stages_for` (`auto`; `::FINAL_ONLY_SEGMENTERS={motion,gemini}`),
`::resolve_video_path` (normalizes collector `./data\..` Windows paths), `::format_aggregate`.
134     - `@backend/eval/harness.py` ? DB-pred scorer (no re-run).
`::STAGES=('candidates','final')`, `::get_predictions`
(candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`); ? unknown
stage raises ValueError; reused by regression.py),
`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate), `::format_report_text`.
135     - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`

```

```

(5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default
0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn` is the substrate plug-in seam.
136 - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid`
clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON `{"model":"logreg"}` tag.
137 - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels**
(PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail
detections aren't FPs ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops
table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid
vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ?
`window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ?
P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate
+ metrics.
138 - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter =
owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean,
spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO.
`::build_frame_arrays`, `::features` (scipy `uniform/maximum_filter1d` windowed; ? speed NOT
fps-normalized yet ? 30 vs 60fps), `::labels_for`, `::probs_to_segments`
(smooth->threshold->merge/min-dur), `::roc_auc` (rank-based, no sklearn), `::run` (per-video AUC
+ LOVO-threshold F1 + oracle-threshold F1). ? DIRECTIONAL only (n?5, linear, threshold-transfer
unsolved); held-out per-frame AUC 0.83?0.92, **beats the heuristic on multi-court footage**
(Kushagra/gBaaddy). Manifest `output/human_lovo_manifest.json` (the n=5 human corpus) shared w/
calibrate_local.
139 - ? **MULTI-COURT / background-game confound (2026-06-08 finding):** WASB tracks EVERY
shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high
dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't
find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally÷dead-
time visibility "contrast" (?3.6× heuristic works, ?1.9× fails). Lever = **foreground-court
isolation** (spatial gate to the prominent court, needs real court-region detection). Academy =
multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
140 - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond
calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT:
duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal),
`::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`),
`::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-
one-video-out needs ?2 videos; final model trained on all.
141 - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT
padded clips (precision lever; eval/tuning, NOT the live pipeline).
`::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (?
FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`,
`::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises
SystemExit); imports `BASELINE` from calibrate_wasb +
`TUNED`/`write_segments_csv`/`seconds_to_mms` from export_wasb_segments.
142 - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,
`::regress_with_provider` (GT via `annotation_registry.get(tier)`;
"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle),
`::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports `annotation_registry` from
**`backend.annotations`** NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass).
143
144 ### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
145 - `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure
numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent`(t,strength),
`::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy
STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k.std`;
`k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2×, band-pass null -> NOT adopted
(PR #35).
146 - `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.

```

```

147
148     ### 2.4 Stitcher / tools / utils
149     - `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::_parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::_get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`(0.5)/`end_padding`(1.0) defaults; temp clips
in a TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break.

```

10. user (2026-06-10T06:13:56.910Z)

39: shadows the build system; update README/`run\_all\_tests.py`/CI references.

11. user (2026-06-10T06:14:25.135Z)

```

backend\config\models.py:41:     wsl_stage_dir: str = "~/clips"
backend\config\models.py:80:     # Directory where the DB, highlight reels, and processed clips
are written.
backend\api\static\index.html:119:                                     <p>Generate seamless high-quality
highlight reel clips compiled directly by FFmpeg.</p>
backend\eval\export_wasb_segments.py:6:segment, by diffing tuned-vs-golden quantitatively, or by
cutting clips.
backend\eval\export_wasb_segments.py:15:     python -m backend.eval.export_wasb_segments
--trajectory ~/clips/sample_long_traj.csv \
backend\eval\export_wasb_segments.py:191:def _maybe_slice(video: str, segments_csv: str,
clips_dir: str) -> None:
backend\eval\export_wasb_segments.py:194:     written = slice_rallies(video, segments_csv,
["all"], clips_dir)
backend\eval\export_wasb_segments.py:195:     print(f"\ncut {len(written)} clip(s) ->
{clips_dir}")
backend\eval\export_wasb_segments.py:214:     ap.add_argument("--video", default=None,
help="source video (with --slice) to cut clips from")
backend\eval\export_wasb_segments.py:216:     ap.add_argument("--clips-dir",
default="output/wasb_tuned_clips")
backend\eval\export_wasb_segments.py:226:     _maybe_slice(args.video, res["out_csv"],
args.clips_dir)
backend\eval\gemini_refine.py:4:EXISTING Gemini provider over the SHORT WASB candidate clips
(not the whole video):
backend\eval\gemini_refine.py:7:nothing (it was a toss / dead time). Cheap because uploads are
seconds-long clips,
backend\eval\gemini_refine.py:114:     tmpdir = osp.join("output", "gemini_refine_clips")
backend\eval\gemini_refine.py:165:     tmpdir = osp.join("output", "gemini_full_clips")
backend\eval\gemini_refine.py:214:     help="downscale uploaded clips to this
width (0=keep source res)")
backend\main.py:283:     parser.add_argument("--output-dir", default="output", help="Directory
where processed clips/highlights are saved")
backend\pipeline\stitcher\compiler.py:84:     # We will create temporary clips and stitch
them using FFmpeg concat demuxer
backend\pipeline\stitcher\compiler.py:85:     temp_clips = []
backend\pipeline\stitcher\compiler.py:154:     temp_clips.append(clip_path)
backend\pipeline\stitcher\compiler.py:173:     if not temp_clips:
backend\pipeline\stitcher\compiler.py:175:         f"[COMPILER ERROR] Failed to
generate ANY valid highlight clips. "
backend\pipeline\stitcher\compiler.py:181:     logger.info(f"Successfully trimmed
{len(temp_clips)}/{len(segments)} clips. Preparing to stitch...")
backend\pipeline\stitcher\compiler.py:184:     list_file_path = os.path.join(tmp_dir,
"clips.txt")
backend\pipeline\stitcher\compiler.py:186:     for clip in temp_clips:
backend\pipeline\stitcher\compiler.py:191:     logger.info("Stitching clips together...")
backend\utils\video.py:29:     Use True for short clips where keyframe drift
matters.
backend\pipeline\detectors\tracknet_runner.py:40:     wsl_stage_dir: str = "~/clips"
# where staged videos/outputs live in WSL
backend\pipeline\detectors\tracknet_runner.py:54:

```

```
wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
backend\tools\rally_slicer.py:1: ""Slice individual rally clips out of a (long) annotated video.
```

[Showing results with pagination = limit: 30]

12. user (2026-06-10T06:14:26.536Z)

```
docs\CODE_MAP.md:31: -> segments = seg.process_video(video_id, video_path, player_pool?,
max_frames?)
docs\CODE_MAP.md:107: - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`, `description`, `process_video`. $ `process_video(video_id, video_path, player_pool=None,
max_frames=None) -> List[dict{id, start_time, end_time, duration, metadata}]`.
docs\CODE_MAP.md:114:- `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name
`'motion'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
docs\CODE_MAP.md:123:[Omitted long matching line]
docs\CODE_MAP.md:246:- **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side
(must be on PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer
host ffmpeg + `--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch +
WASB repo modules).
```

13. user (2026-06-10T06:14:59.192Z)

Structured output provided successfully

14. assistant (2026-06-10T06:15:05.153Z)

Done.